AS OF 29 OCTOBER 2026 · WRITTEN BY JILL MORENO

Contributing to the dashboard

PART 0

What this is

This page gets you from nothing to your first change in front of the team. Read it once, on your first day. After that, the day-to-day guide is the one you keep open.

There are only three things to understand, and the rest of this page is detail about them.

The Hub

A website where everyone — designers, developers, stakeholders — sees the team's work. It has a list of the product as it stands, and a board where work under discussion is reviewed.

Your branch

Your own private copy of the project. You change it as much as you like. Nothing on it is seen by anyone until you choose to share it, and nothing on it changes the product until Jill decides it does.

Claude

The desktop app you type sentences to. It does every technical step for you — copying the project, saving your work, publishing it — so you never type a command.

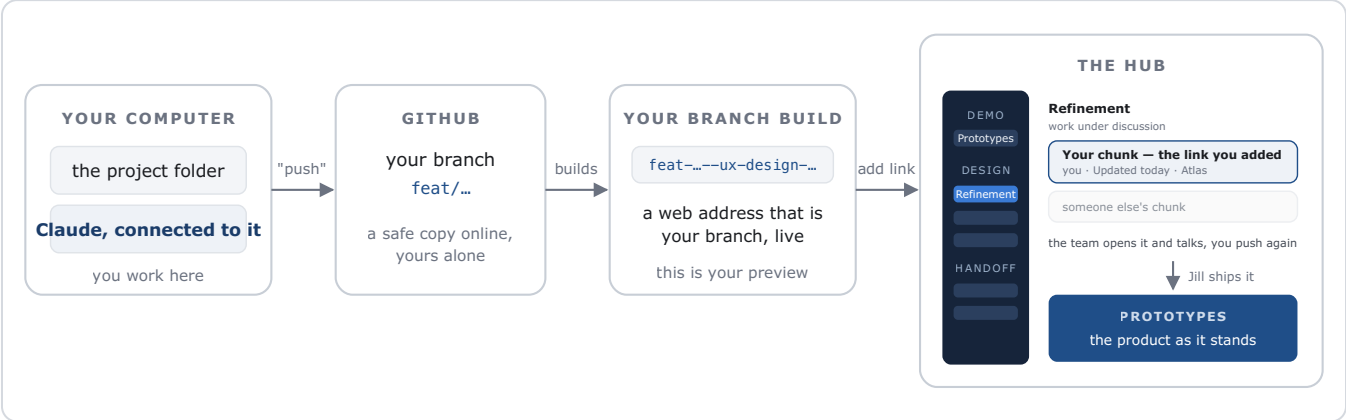
You design. Claude does the technical parts. Jill decides what ships.

You never type a command, you never run the website on your own computer, and you never touch the copy of the project that everyone sees.

PART 1

The whole thing in one picture

Your work travels left to right. Every box is a place; every arrow is one sentence you say to Claude, or one click.



In words, left to right:

1. **You change the project** in a folder on your computer, by talking to Claude.
2. **Claude sends your branch to GitHub** when you say "push". GitHub is the online copy; your branch there is yours alone.
3. **Netlify builds your branch into a website** at its own address, a few minutes after every push. That address is your preview. You never run the site yourself.
4. **You add that address to Refinement** on the Hub. The moment you do, everyone with the Hub's link can open it.
5. **The team looks and talks.** You change, push again; the address stays the same.
6. **Jill ships it.** When a piece is decided, she puts your branch into the product. Prototypes then shows it, and you start the next piece on a fresh branch.

Most Important Thing.
You work on YOUR BRANCH. Never push directly to main.

"Push" always means your branch. Main is the copy everyone sees, and only Jill puts anything there. If you ever find yourself on main, stop and tell her.

> **The one-sentence version.** Branch, build, share, discuss, ship. The day-to-day guide walks that loop every day; this page gets you to the start of it.

PART 2

What you need

Four things, and Jill gives you three of them.

YOU NEED	HOW YOU GET IT	HOW YOU KNOW IT WORKED
A GitHub account, invited to the project	Make a free account at github.com if you have none. Jill invites you; GitHub emails you; click **Accept**.	Open `github.com/jillmoreno/xcel-ux-dashboard`. A list of files means you are in; a "404" means ask Jill.
The Claude desktop app, connected to GitHub	Install Claude. Inside it: **Settings** → **Connectors** → **GitHub** → **Connect**, sign in when it asks, and allow it.	GitHub shows as connected in Settings. This is what lets Claude copy the project down and send your work up.
An empty folder on your computer	Make one anywhere, for example *Documents › xcel-ux-dashboard*. Claude fills it in Part 3.	It exists and it is empty.
The Hub's address and password	Ask Jill. Every branch build uses the same password.	The Hub opens in your browser.

> **You do not need** Terminal, Node, git, GitHub Desktop, a Netlify account, or any idea what those are. Anything that would normally need a typed command, Claude does when you ask in plain English.

PART 3

Setting up your computer

Once, on your first day. Four sentences to Claude, each with a copy button when you hover it.

1 **Connect the folder.**

Open the Claude desktop app, go to the **Code** tab, and choose the empty folder you made. From now on, Claude can read and change what is in it.

2 Copy the project down.

YOU

> Clone the GitHub repository jillmoreno/xcel-ux-dashboard into this folder.

"Clone" means copy. A minute later the folder is full of the project's files. You will not read them; Claude will.

3 Install what the project needs.

YOU

> Install the project's dependencies.

The project leans on a few hundred small pieces of code other people wrote. This fetches them. A few minutes; you only do it once, and Claude will tell you if it ever needs doing again.

4 Put your name on your work.

YOU

> For this project, set my git name to Your Name and my git email to you@scorebuilders.com.

Use your real name and your work email. Every change you save carries this name, so the team can see whose it is. Skip it and your work shows up as "Claude".

5 Check.

YOU

> Which branch am I on, and is the project up to date?

Claude should say you are on `main` and up to date. That is the one time you should be on main — Part 7 moves you off it before you change anything.

> **Nothing here runs the website.** You will never start the site on your own computer, and you do not need to. Your preview is the branch build in Part 1, which Netlify makes for you.

PART 4

The words you will see

You never type these. Claude says them back to you, so here is what each one means.

WORD	PLAIN MEANING
the Hub	The website where the team's work is seen. One address, one password, everyone.
main	The one copy of the project that IS the product. Only Jill changes it.
branch	Your own copy of the project, named like `feat/study-streak-card`. Everything you do happens on a branch.
commit	A saved snapshot of your branch, with a short note saying what changed. Several a day is normal.
push	Sending your commits to GitHub. After every push, Netlify builds your branch and the team can see it.
branch build	Your branch as a live website, at its own address. Your preview, and the thing you share.
rebase	Moving your branch so it starts from today's main instead of the day you made it. Every few days; the day-to-day guide shows how.
ship to main, merge	Jill putting your branch into main. The only way anything reaches the product.
switch, feature flag	A named on/off in the code that hides your change until it is decided. Part 6.
Refinement	The Hub's board for work under discussion. Anyone adds to it, on the page.
Prototypes	The Hub's list of the product as it stands. Only shipping changes it.

WORD	PLAIN MEANING
routine	A ready-made checklist Claude runs with you, started by typing `/` and its name. `/share-branch` is the one you will use most.

> **You work on your branch. Jill works on main.** The one rule under every word above — it is the stop sign in Part 1.

PART 5

Where your work lives

Two kinds of work, two homes. Picking the wrong one is the most common mistake here, so say which kind it is when you ask Claude to build it.

YOU ARE MAKING	SAY TO CLAUDE	WHAT IT IS	REVIEWERS OPEN
A piece of the product	"a React component, behind a feature flag that is on by default on this branch"	A page, a card, a panel, or a new version of something that already exists, in the app's own code.	Your branch build at `/dashboard-rebrand?demo=1`
A standalone page to look at	"a standalone HTML page in public/demos"	A single hand-made web page, not wired into the app. An idea, a comparison, a brief answered in pictures.	Your branch build at `/demos/your-file.html`

> **Never `public/prototypes/`.** That folder holds finished explorations that each have a row on the Hub and their own tests. Work in review goes in `public/demos/`, which exists for exactly that.

Changing something that already exists? Do not edit it. Ask Claude to copy it to a sibling and build on the copy, with a switch choosing between the two. That way you and the person editing the original never collide. The routine `/promote-component` does this for you.

PART 6

The one rule: switches

Every change to the product sits behind a switch. That one idea is what lets five designers work on one product without anyone's half-finished work appearing in front of stakeholders.

1. **On your branch, the switch is on.** Your branch build shows your work.
2. **On main, the switch is off.** The product looks exactly as it did.
3. **When Jill ships your branch, she decides the switch.** On, and it is in the product. Off, and the code is there but nothing shows until someone decides otherwise.

You get all of this by asking for it in the same breath as the work:

YOU

> Build it behind a feature flag that is on by default on this branch.

Then ask "What is the flag called?" and write it down. Reviewers can flip any switch live on your build from the robot icon on the dark bar, so you can show both versions from one address.

> **Why not just add it to Prototypes?** Prototypes holds one row per surface, all pointing at main, and a test refuses any other row. Your work reaches Prototypes by its switch turning on in main, never by adding a row.

PART 7

Your first change, start to finish

Set up and ready? This is the loop from Part 1 with the actual sentences. The day-to-day guide has the long version; this is enough for day one.

1 **Get off main and onto a branch of your own.**

YOU

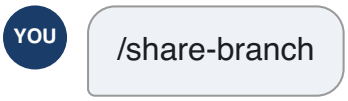
> Fetch origin, then make a new branch called feat/short-name from origin/main and switch to it.

Name it `feat/` and two or three words, lowercase, joined with dashes. Claude confirms you are on it.

2 **Say what you want, the way you would to a developer beside you.**

Name the page, the component, what it should do instead, and which kind of work it is (Part 5). Ask for the switch (Part 6). If Claude names a file you do not recognise, ask "What is that file?" before saying yes.

3 **Share it.**



One routine does the rest: it checks the work, saves it, pushes it, waits for Netlify to build your branch, and opens the Refinement form already filled in. You read it over and click **Add link**. Then it opens a card for the To Do Board; click **Add card**.

4 **Look at it the way a reviewer would.**

Open your row on the Hub and click through from the page a reviewer would start on, not straight to your component. Things that are invisible on a direct link are obvious on the first click.

5 **Keep going.**

Each further change is step 2 and then `/share-branch` again; the address stays the same. When the next change is really a different piece of work, start a fresh branch for it.

> **That is a chunk.** Finish a chunk, share it, start a fresh branch for the next. Small branches are the habit the whole Hub is built around.

PART 8

Things never to do

Short, because Claude refuses most of them for you and says why.

- **Never push to main.** Every push to main rebuilds the Hub for everyone. Be on your branch (Part 7, step 1) before asking Claude to push anything. You cannot do it by accident: Claude refuses the command and says whose step it is, and GitHub refuses the push even if something slipped past Claude.
- **Never add a row to Prototypes.** Your work gets there by its switch turning on in main, nothing else.
- **Never run the site on your own computer.** Your branch build is the preview.

- **Never edit the Hub's own files.** The project list, the demo bar, the routines. Claude refuses and names the file; ask Jill.
- **Never put review pages in `public/prototypes/`.** Those go in `public/demos/`.
- **Never invent a number.** A fee, a lesson count, a date XCEL has not published is marked as invented in the code and on screen. Ask instead.
- **Never share the main Hub address as your work.** Send your branch build. The main address shows what is already decided, and everything else the team is doing.

PART 9

Then: the day-to-day guide

That is the setup done, and your first change shared. Everything after today is the other guide.

It covers sitting down each day, keeping your branch current, building a change properly, sharing it on Refinement, making your surface the prototype, writing the dev handoff for engineering, the ready-made routines, the morning auto-rebase, and what to say to Claude when something goes wrong.

Designer Workflow on the Hub →

The two guides are one shape on purpose: this one is **once, your first day**; that one is **every day after**. The glossary and the troubleshooting table live there.

Contributing to the dashboard · 29 October 2026 · The first-day guide. Every day after is the Designer Workflow guide; stakeholders get How to read this dashboard.